

PRACTICE EXERCISES



Module 3: Advanced SQL Queries

Module 3 - 8 Hours

Advanced SQL Queries

Instructions

Work against `icicibank` with the full dataset: 8 branches, 30 customers, 36 accounts, 495 transactions, 12 loans, 144 loan payments, 24 employees, 20 fixed deposits.

Level	Meaning	Expected time
Level 1	Recall and understanding	25 min
Level 2	Apply. Write and run SQL.	70 min
Level 3	Analyse and decide	35 min

Rules:

- Include the SQL **and** the output for every query answer.
- Where a question asks for a row count, state it explicitly.
- Do not modify the data. These are all read queries.

How much to attempt

The questions marked **(Core)** are the required set. They fit the time above, and together they cover every objective of the module. Everything unmarked is **stretch work**: attempt it if you finish early, or take it home.

You are not behind if you have completed only the core set.

Level	Core set	Stretch
Level 1	Q1, Q2, Q3, Q6, Q7, Q8, Q9	Q4, Q5, Q10
Level 2	Q11, Q12, Q13, Q18, Q20, Q22, Q29	Q14, Q15, Q16, Q17, Q19, Q21, Q23, Q24, Q25, Q26, Q27, Q28
Level 3	Q30, Q32	Q31, Q33, Q34

Level 1 - Understanding

Q1. (Core) A query joins `branch`, `customer`, `account`, `account_txn` and `card`. How many join conditions does it need, and what happens if you supply one fewer?

Q2. (Core) Explain the difference between these two queries in one sentence each, and state which returns more rows.

```
SELECT c.customer_id FROM customer c JOIN account a ON a.customer_id = c.customer_id;
SELECT c.customer_id FROM customer c LEFT JOIN account a ON a.customer_id = c.customer_id;
```

Q3. (Core) This query was meant to list every branch with its overdue loans, including branches that have none. It returns only branches that have overdue loans. Explain the defect and write the fix.

```
SELECT b.branch_name, l.loan_id
FROM   branch b
       LEFT JOIN loan l ON l.branch_id = b.branch_id
WHERE  l.status = 'OVERDUE';
```

Q4. Write the general shape of an anti-join in words, and state which column you must test for NULL and why it must not be a nullable column.

Q5. Why does the ICICI Bank employee hierarchy query need a `LEFT JOIN` rather than an inner join? What exactly is lost if you use an inner join?

Q6. (Core) For each, say whether the value counts NULLs, and give the number you would expect against `account`.

1. `COUNT(*)`
2. `COUNT(interest_rate)`
3. `COUNT(DISTINCT account_type)`
4. `AVG(balance)`

Q7. (Core) State the `GROUP BY` rule in one sentence, then explain why the engine cannot simply pick a value for an ungrouped column.

Q8. (Core) Complete the table.

	WHERE	HAVING
Filters what	?	?
Runs when	?	?
Can reference aggregates	?	?
Which should you prefer, and why	?	

Q9. (Core) This query returns zero rows even though there are clearly customers who are not nominees. Explain exactly why, and give two different fixes.

```
SELECT COUNT(*) FROM customer c
WHERE  c.first_name NOT IN (SELECT nominee_name FROM fixed_deposit);
```

Q10. Explain what "fan-out" means in a join, and describe a banking report where it would produce a wrong but believable number.

Level 2 - Applying SQL

Joins

Q11. (Core) List every card in the bank with the account number it belongs to, the customer's full name, and the branch name. Show card type, masked number and status. Sort by branch name then customer name.

Q12. (Core) List all customers at Koramangala and Whitefield with their account numbers and balances, richest first. Include customers who hold no account.

Q13. (Core) Find every account that has never had a transaction, showing account number, account type, status and the owning customer's name.

Q14. Find every account that has no card issued against it. Show account number, account type and the customer's name. Give the count and the list.

Q15. Produce the full ICICI Bank staff list showing employee name, designation, branch name, and the name of their manager. Branch Managers must appear with a blank or a label rather than being dropped.

Q16. For each Branch Manager, show their name, their branch, the size of their team, and the total salary of their team excluding their own.

Q17. Build a `loan_risk_band` table with the bands below, then use a non-equi join to report the number of loans and total outstanding in each band.

band_code	band_name	min_outstanding	max_outstanding
L1	Small	0	500000
L2	Medium	500000	2000000
L3	Large	2000000	5000000
L4	Very Large	5000000	99999999

Aggregates

Q18. (Core) For each branch, report the number of customers, number of accounts, total balance and average balance. Include branches with no customers. Sort by total balance descending.

Q19. For each loan type, report the count of loans, total principal disbursed, total outstanding and the percentage of principal still outstanding, rounded to one decimal place.

Q20. (Core) For each channel, report the number of successful transactions, the number of failed transactions, the total successful value and the failure rate as a percentage rounded to two decimals. Sort by failure rate descending.

Q21. Find every branch whose total outstanding loan book exceeds 2,000,000. Show branch name, loan count and total outstanding.

Q22. (Core) For each customer holding at least two accounts, show the customer name, the number of accounts and their combined balance. Sort by combined balance descending.

Q23. Produce a monthly transaction summary: for each calendar month in the data, show the month, the number of transactions, total credits, total debits and the net movement. Sort by month.

Subqueries

- Q24.** List every account whose balance is more than twice the bank-wide average balance.
- Q25.** For each branch, show the branch name and the balance of its single largest account, using a correlated subquery.
- Q26.** List every customer who holds a loan but no fixed deposit. Write it twice: once with `NOT EXISTS` and once with a `LEFT JOIN` anti-join. Confirm both return the same rows.
- Q27.** Find the customers whose total transaction value this quarter is above the average total across all customers. Use a derived table.

Set operators

- Q28.** Produce a single list of every ICICI Bank product holding, with columns `customer_id`, `product_type` and `amount`, drawn from `fixed_deposit`, `loan` and `account`. Sort by customer then product type.
- Q29. (Core)** Using set operators, answer all three: customers holding both a deposit and a loan; customers holding a deposit but no loan; customers holding a loan but no deposit. State the count for each and confirm the three numbers are consistent with the total number of customers holding at least one of the two products.

Level 3 - Analysis and decisions

Q30. (Core) A colleague sends you this query for the monthly pack and says the customer numbers "look a bit high". Identify every defect and rewrite it correctly.

```
SELECT b.branch_name,
       COUNT(c.customer_id) AS customers,
       SUM(a.balance)      AS total_deposits,
       SUM(t.amount)       AS total_transacted
FROM   branch b
       JOIN customer c   ON c.home_branch_id = b.branch_id
       JOIN account a    ON a.customer_id    = c.customer_id
       JOIN account_txn t ON t.account_id     = a.account_id
WHERE  t.txn_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31'
GROUP BY b.branch_name;
```

Q31. The risk team wants "the top three customers by balance in each branch". Explain why `GROUP BY` and `HAVING` alone cannot answer this, and describe what you would need. Name the module of this programme that provides it.

Q32. (Core) A report using `NOT IN` against a subquery has been returning zero rows for six weeks and nobody noticed, because the report is a compliance exception list and zero exceptions looked like good news. Write four sentences on what this tells you about designing checks for reports whose correct answer is often zero.

Q33. The branch performance pack takes 40 seconds against 40 million transaction rows and the operations head runs it every morning at 9 a.m., when the branches open. Without changing the SQL, list three things you would do. For each, name the module of this programme that covers it.

Q34. ICICI Bank is merging with a smaller bank. You are asked to prove that every transaction in the acquired bank's extract has landed in ICICI's core system, and that ICICI has not gained any transaction that was not in the extract. Describe the exact SQL approach, say what a non-empty result means in each direction, and state one reason a row could differ that is **not** a missing row.

Q35. (Written, 10 to 15 lines.) Reply to ticket ICICI-4530. Tell the Head of Branch Operations what the pack now shows, name two specific findings from the data you have just queried, and state one limitation of the report that they should know about before acting on it.